

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Develop a Student Portal Using JavaScript Classes and DOM Manipulation

2. AIM

To design and develop a Student Portal web page that uses an ES6 JavaScript class, form input capture, and DOM manipulation to collect student details and display them dynamically on the page.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use ES6 class syntax to model a Student object.
 - To capture user input from HTML form fields using `getElementById().value`.
 - To create an object instance and pass it to a function for further processing.
 - To use `innerText` to dynamically update DOM elements with object data.
 - To structure a two-panel layout (input card and display card) using CSS Flexbox.
 - To style form elements, buttons, and cards with CSS3 (border-radius, transitions, hover and focus effects).
 - To build a responsive layout that stacks vertically on smaller screens using a media query.
-

4. THEORY

ES6 Classes provide a template for creating objects with shared structure. The `constructor()` method runs when a new instance is created with the `new` keyword, and this refers to the instance being built.

DOM Selection and Manipulation — `document.getElementById()` retrieves a reference to an element by its id; `.value` reads the current text of an input field, and `.innerText` sets the visible text content of an element.

Event Handling — the `onclick` attribute runs a function when a button is clicked; `event.preventDefault()` on the form's `onsubmit` stops the browser's default page-reload behaviour.

CSS Flexbox Layout — `display: flex` on the `.container` arranges the input card and display card side by side, with `gap` controlling the spacing between them.

Responsive Design — a `@media (max-width: 600px)` query changes `flex-direction` to `column` so the two cards stack vertically on narrow screens.

5. ALGORITHM / PROCEDURE

- Create three files: `index.html`, `style.css`, and `script.js`.
 - In `index.html`, link the stylesheet in `<head>` and the script before the closing `</body>` tag.
 - Build a `<main class="container">` holding two `<section class="card">` elements.
 - In the first card, add a form with input fields for Full Name, Roll Number, Department, and CGPA.
 - Add a Submit button that calls `display(createstu())` on click.
 - In the second card, add labelled `<span>` elements with ids to hold the displayed name, roll number, department, and CGPA.
 - In `script.js`, define a `Student` class with a constructor that stores `name`, `rn`, `dept`, and `cgpa`.
 - Write `createstu()` to read all four input values with `getElementById().value` and return a new `Student` object.
 - Write `display(stu)` to set the `innerText` of each output `span` from the given `Student` object's properties.
 - In `style.css`, reset default spacing, set the font stack, and centre the page content.
 - Style `.container` with `display: flex` and `gap`; style `.card` with padding, border-radius, and a border.
 - Style inputs with padding, border-radius, and a focus border-colour transition.
 - Style the button with a dark background, rounded corners, and a hover background-colour transition.
 - Add a `@media (max-width: 600px)` rule to stack the cards vertically on narrow screens.
 - Open `index.html` in a browser, fill the form, click Submit, and verify the display card updates.
 - Record expected vs. actual output for each test case.
-

6. PROGRAM

`index.html`

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Portal</title>
```

```
<link rel="stylesheet" href="style.css">
</head>
<body>

  <main class="container">

    <!-- Input Form -->
    <section class="card">
      <h2>Enter Student Details</h2>

      <form id="student-form" onsubmit="submitStudent(); return false;">

        <div class="field">
          <label for="iname">Full Name</label>
          <input
            type="text"
            id="iname"
            placeholder="e.g. Ramu"
            required
          >
        </div>

        <div class="field">
          <label for="irn">Roll Number</label>
          <input
            type="text"
            id="irn"
            placeholder="e.g. 101"
            required
          >
        </div>

        <div class="field">
          <label for="idept">Department</label>
          <input
            type="text"
            id="idept"
            placeholder="e.g. Computer Science"
            required
          >
        </div>

        <div class="field">
          <label for="icgpa">CGPA</label>
          <input
```

```

        type="number"
        id="icgpa"
        placeholder="e.g. 8.9"
        min="0"
        max="10"
        step="0.01"
        required
    >
</div>

<button type="submit">Submit</button>

</form>
</section>

<!-- Output Display -->
<section class="card display-card">
    <h2>Student Details</h2>

    <div class="info-group">
        <span class="label">Name</span>
        <span id="uname" class="value">—</span>
    </div>

    <div class="info-group">
        <span class="label">Roll No</span>
        <span id="urn" class="value">—</span>
    </div>

    <div class="info-group">
        <span class="label">Department</span>
        <span id="udept" class="value">—</span>
    </div>

    <div class="info-group">
        <span class="label">CGPA</span>
        <span id="ucgpa" class="value">—</span>
    </div>

</section>

</main>

<script src="script.js"></script>

```

```
</body>
</html>
```

style.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}
```

```
body {
  background-color: #f7f7f8;
  color: #1a1a1a;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  padding: 20px;
}
```

```
.container {
  display: flex;
  gap: 24px;
  width: 100%;
  max-width: 720px;
}
```

```
.card {
  flex: 1;
  background: #ffffff;
  padding: 28px;
  border-radius: 12px;
  border: 1px solid #e5e5e7;
}
```

```
h2 {
  font-size: 1.1rem;
  font-weight: 600;
  margin-bottom: 20px;
  color: #111111;
}

.field {
  margin-bottom: 16px;
}

label, .label {
  display: block;
  font-size: 0.75rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: #6e6e73;
  margin-bottom: 6px;
}

input {
  width: 100%;
  padding: 10px 12px;
  font-size: 0.9rem;
  border: 1px solid #d2d2d7;
  border-radius: 6px;
  outline: none;
  transition: border-color 0.15s ease;
}

input:focus {
  border-color: #0066cc;
}
```

```
button {  
  width: 100%;  
  margin-top: 8px;  
  padding: 10px;  
  background-color: #111111;  
  color: #ffffff;  
  font-size: 0.875rem;  
  font-weight: 500;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  transition: background-color 0.15s ease;  
}
```

```
button:hover {  
  background-color: #333333;  
}
```

```
.display-card {  
  background-color: #fafafa;  
}
```

```
.info-group {  
  margin-bottom: 16px;  
  padding-bottom: 10px;  
  border-bottom: 1px solid #eeeeef;  
}
```

```
.info-group:last-child {  
  border-bottom: none;  
  margin-bottom: 0;  
}
```

```
.value {  
  font-size: 0.95rem;
```

```
font-weight: 500;
color: #1d1d1f;
}

@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }
}
```

script.js

```
// Student class to create Student objects
class Student {

  // Constructor initializes student properties
  constructor(name, rn, dept, cgpa) {
    this.name = name;
    this.rn = rn;
    this.dept = dept;
    this.cgpa = cgpa;
  }
}

// Function to create a Student object from user input
function createstu() {
  // Get values entered in the form
  const name = document.getElementById("iname").value;
  const rn = document.getElementById("irn").value;
  const dept = document.getElementById("idept").value;
  const cgpa = document.getElementById("icgpa").value;

  // Create and return a new Student object
  return new Student(name, rn, dept, cgpa);
}

// Function to display student details on the webpage
function display(stu) {
  document.getElementById("uname").textContent = stu.name;
  document.getElementById("urn").textContent = stu.rn;
  document.getElementById("udept").textContent = stu.dept;
  document.getElementById("ucgpa").textContent = stu.cgpa;
}
```

```
// Function called when Submit button is clicked
function submitStudent() {
    // Create Student object using input values
    const stu = createstu();
    // Display the student's details
    display(stu);
}
```

7. CODE EXPLANATION

Code	Explanation
class Student	Defines a blueprint object holding name, roll number, department, and CGPA.
constructor()	Initializes each new Student instance with the provided values.
getElementById().value	Reads the current value typed into a form input field.
createstu()	Reads all four form fields and returns a new Student object.
display(stu)	Updates the output spans on the page with fields from the given Student object.
onclick="display(createstu())"	Chains input capture and DOM update together in a single click handler.
innerText	Sets the visible text content of a DOM element.
event.preventDefault()	Stops the form's default submission/page-reload behaviour.
display: flex (.container)	Places the input card and display card side by side.
border-radius	Rounds the corners of the cards, inputs, and button.
button:hover	Darkens the button background on mouse-over.
input:focus	Highlights the input border colour when the field is active.
@media (max-width: 600px)	Stacks the two cards vertically on narrow screens.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load the page in a browser	Input card and display card render side by side	Both cards displayed side by side correctly	Pass
TC-02	Leave all fields blank and click Submit	The Browser Shows the Error	Browser displays that Please fill the Fields	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-03	Enter valid name, roll no, department, and CGPA, then click Submit	Display card updates with the entered values	All four values displayed correctly	Pass
TC-04	Enter a CGPA above 10	Number input restricts or flags the out-of-range value	Browser shows error that the Input must be between 0 to 10	Pass
TC-05	Click Submit again with different values	Display card updates to show the latest values	Display card refreshed with new values	Pass
TC-06	Focus on an input field	Input border colour changes to blue	Border colour changed on focus	Pass
TC-07	Hover over the Submit button	Button background darkens	Background colour darkened smoothly	Pass
TC-08	Resize the browser to a narrow width	The two cards stack vertically	Cards stacked into a single column	Pass

9. OUTPUT

Output 1: Initial Page Load

The page renders the input card and the display card side by side. The display card shows a dash (—) placeholder for each field before any data is submitted.

Enter Student Details

FULL NAME

e.g. Ramu

ROLL NUMBER

e.g. 101

DEPARTMENT

e.g. Computer Science

CGPA

e.g. 8.9

Submit

Student Details

NAME

—

ROLL NO

—

DEPARTMENT

—

CGPA

—

Output 2: Submit Unfilled Registration Form

The input card shows the Blank fields if we don't fill them and Submit the Student Details.

The screenshot shows two side-by-side cards on a light gray background. The left card, titled 'Enter Student Details', contains four input fields: 'FULL NAME' (with placeholder 'e.g. Ramu'), 'ROLL NO' (with placeholder 'e.g. 101'), 'DEPARTMENT' (with placeholder 'e.g. Computer Science'), and 'CGPA' (with placeholder 'e.g. 8.9'). A red error message box with an exclamation mark icon is positioned over the 'ROLL NO' field, displaying the text 'Please fill out this field.' Below the fields is a black 'Submit' button. The right card, titled 'Student Details', displays the same four fields but with empty lines instead of text, indicating the form is unfilled.

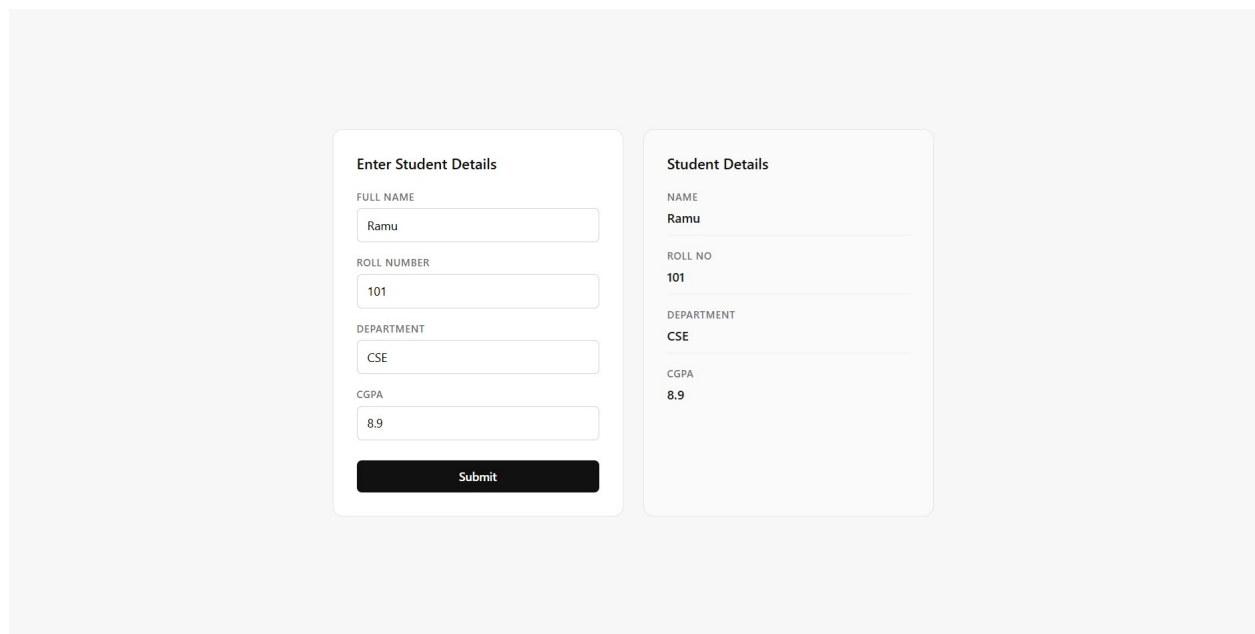
Output 3: Filled Registration Form

The input card shows the Full Name, Roll Number, Department, and CGPA fields filled in with sample student data.

This screenshot shows the same two cards as in Output 2. The 'Enter Student Details' card on the left now has the input fields filled with sample data: 'FULL NAME' contains 'Ramu', 'ROLL NUMBER' contains '101', 'DEPARTMENT' contains 'CSE', and 'CGPA' contains '8.9'. The 'Submit' button remains. The 'Student Details' card on the right remains empty, showing only the field labels and empty lines.

Output 4: Display Card After Submit

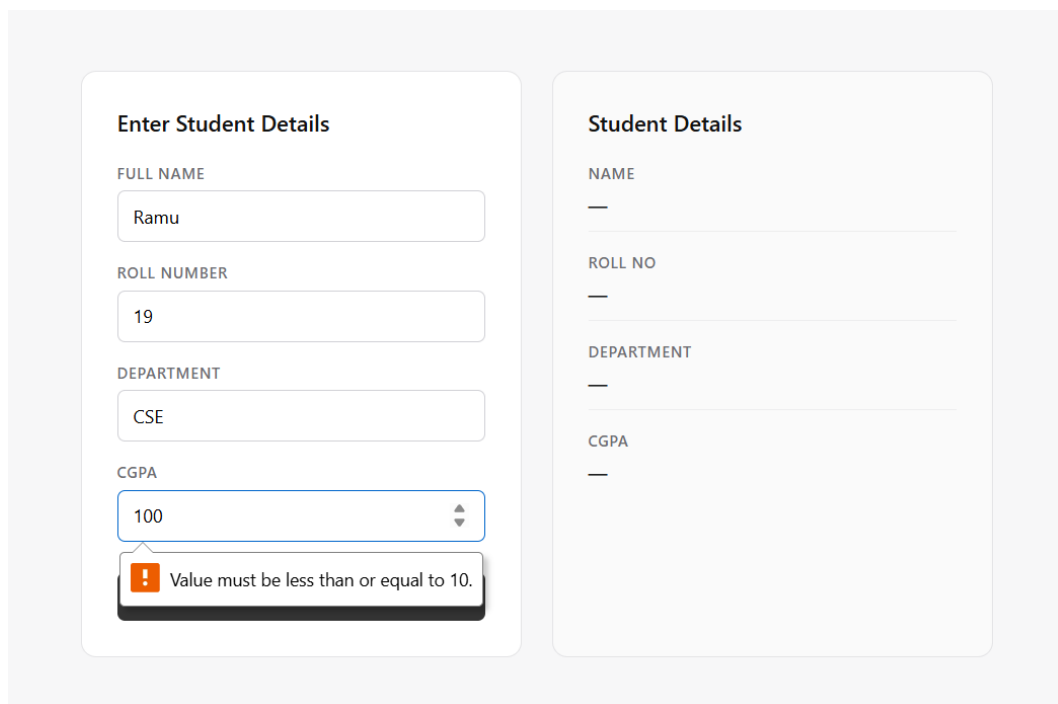
After clicking Submit, the display card updates to show the entered name, roll number, department, and CGPA.



The image shows two side-by-side cards on a light gray background. The left card, titled "Enter Student Details", contains four input fields: "FULL NAME" with the value "Ramu", "ROLL NUMBER" with "101", "DEPARTMENT" with "CSE", and "CGPA" with "8.9". A black "Submit" button is at the bottom. The right card, titled "Student Details", displays the submitted information: "NAME" as "Ramu", "ROLL NO" as "101", "DEPARTMENT" as "CSE", and "CGPA" as "8.9".

Output 4: Display Error If CGPA > 10

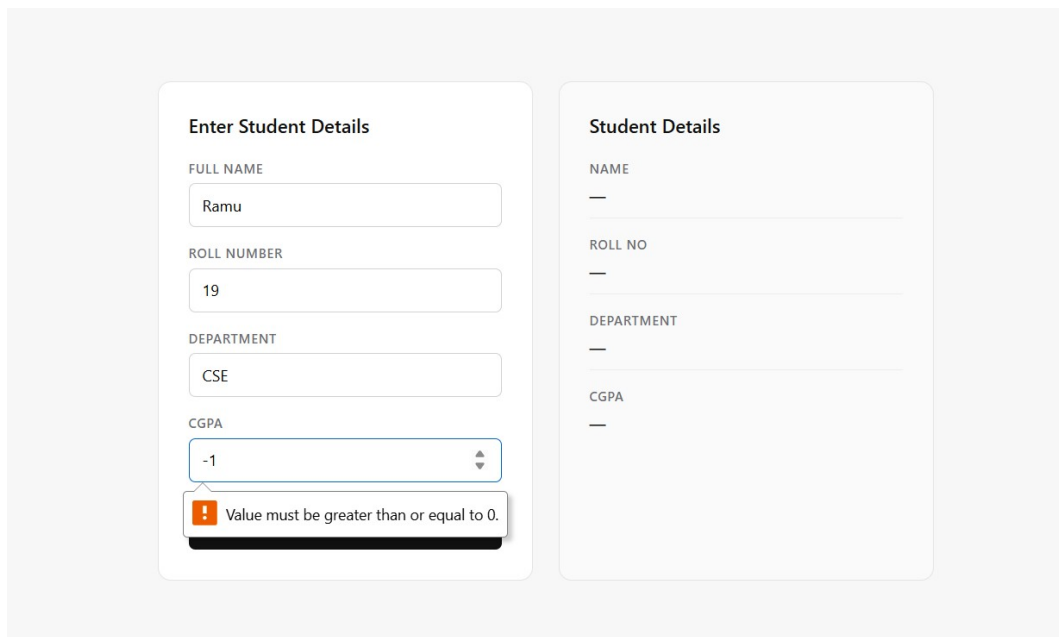
If the CGPA is Greater than 10 , it will show the Error that “The Value must be less than or equal to 10”.



The image shows the "Enter Student Details" form with the following values: "FULL NAME" is "Ramu", "ROLL NUMBER" is "19", "DEPARTMENT" is "CSE", and "CGPA" is "100". The CGPA input field is highlighted with a blue border. Below it, a red error message box with an exclamation mark icon displays the text: "Value must be less than or equal to 10." The "Student Details" card on the right is empty, showing only the labels for the fields.

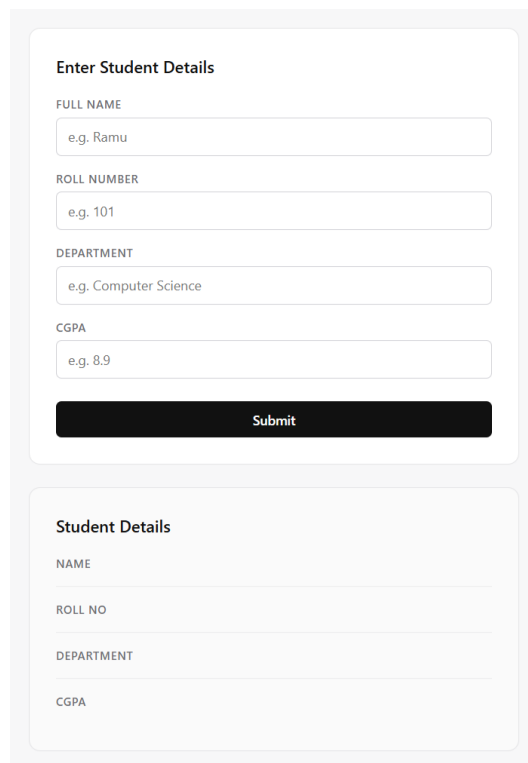
Output 5: Display Error If CGPA < 0

If the CGPA is less than 0 , it will show the Error that “The Value must be greater than or equal to 0”.



The image shows a web interface with two panels. The left panel, titled "Enter Student Details", contains four input fields: "FULL NAME" with the value "Ramu", "ROLL NUMBER" with the value "19", "DEPARTMENT" with the value "CSE", and "CGPA" with the value "-1". Below the CGPA field, a red error message box is displayed, stating "Value must be greater than or equal to 0." The right panel, titled "Student Details", shows the same four fields but with empty input lines, indicating the data has been submitted or is being previewed.

Output 5: Browser Resize Output



The image shows a web interface with two panels. The left panel, titled "Enter Student Details", contains four input fields: "FULL NAME" with the value "e.g. Ramu", "ROLL NUMBER" with the value "e.g. 101", "DEPARTMENT" with the value "e.g. Computer Science", and "CGPA" with the value "e.g. 8.9". Below the CGPA field, there is a black "Submit" button. The right panel, titled "Student Details", shows the same four fields but with empty input lines, indicating the data has been submitted or is being previewed.

10. RESULT

Thus, a Student Portal web page was successfully designed and developed using an ES6 JavaScript class, form-based input capture, and DOM manipulation. The page demonstrates object-oriented structuring of data, dynamic display updates, and a responsive two-card Flexbox layout with CSS3 styling. All eight test cases passed successfully, confirming correct data capture, display, and responsive behaviour.